CERTIFICATE

This is to certify that the project report entitled "**Dishcovery**: A Recipe Discovery and Ingredient Matching System" is the bonafide work of Ashita Shrivastava, carried out in partial fulfilment of the requirements for the Bachelor of Computer Applications (BCA) in Data Science at Chandigarh University, under the guidance and supervision of the undersigned.

The project demonstrates the design and implementation of a database-driven web application that stores recipe information, ingredients, categories, and recipe–ingredient relationships in a MySQL database. The application provides ingredient-based recipe matching and displays recipe details through a web interface.

TABLE OF CONTENTS

Abstract

Graphical Abstract

Abbreviations

Symbols

CHAPTER 1. INTRODUCTION

1.1 Identification of Need / Contemporary Issue

1.2 Identification of Problem

1.3 Identification of Tasks

1.4 Timeline

1.5 Organization of the Report

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Problem

2.2 Existing Solutions

2.3 Comparative Feature Analysis

2.4 Review Summary

2.5 Problem Definition

2.6 Goals / Objectives

CHAPTER 3. DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Features

3.2 Design Constraints

3.3 Feature Finalization

3.4 Design Alternatives and Flow

3.5 Design Selection

3.6 Implementation Plan / Methodology

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

5.2 Future Work

References

Appendix

User Manual

ABSTRACT

Dishcovery is a database-driven web application designed to help users discover recipes based on ingredients available in their kitchens. People often decide what to cook without having a clear view of which dishes can be prepared from the ingredients they already possess. This can lead to unnecessary shopping, repeated meals, and difficulty planning simple meals. Dishcovery addresses this problem by allowing users to select available ingredients and retrieve recipes according to ingredient matches.

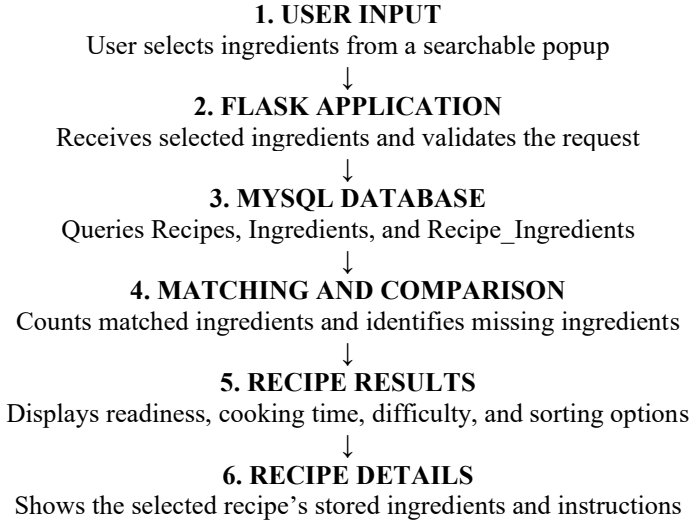
The application uses Flask for server-side processing, MySQL for structured data storage, and HTML, CSS, and JavaScript for the user interface. The database separates recipe records, ingredient records, category records, and the relationships between recipes and ingredients. This relational structure supports organized storage, reuse of ingredient data, and queries across related tables.

Users can select ingredients through a searchable popup, submit them for matching, and view recipes with the number of required ingredients available, missing ingredients, cooking time, difficulty, and a readiness label. Search results can be sorted by cooking time, the number of available ingredients, or the number of missing ingredients. A separate recipe details page provides the stored cooking instructions and ingredient information.

The project demonstrates practical DBMS concepts including relational schema design, primary and foreign key relationships, joins, filtering, sorting, and database-backed web application development. The current implementation provides a foundation that can later be extended with user accounts, dietary filters, recipe ratings, and more advanced recommendation methods.

Keywords: DBMS, MySQL, Flask, recipe discovery, ingredient matching, relational database, web application.

GRAPHICAL ABSTRACT



ABBREVIATIONS

Abbreviation	Meaning
DBMS	Database Management System
SQL	Structured Query Language
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	Hypertext Transfer Protocol
CRUD	Create, Read, Update, Delete
PK	Primary Key
FK	Foreign Key
API	Application Programming Interface
ER	Entity–Relationship

SYMBOLS

Symbol / Term	Meaning in this report
recipe_id	Unique identifier for a recipe record
ingredient_id	Unique identifier for an ingredient record
category_id	Unique identifier for a category record
matched_count	Number of recipe ingredients present in the user's selection
total_ingredients	Total number of ingredients associated with a recipe
missing_count	Number of recipe ingredients not present in the selection
→	Indicates the direction of process flow
PK	Primary key used to identify a record uniquely
FK	Foreign key used to link related records

CHAPTER 1: INTRODUCTION

1.1 Identification of Need / Relevant Contemporary Issue

Digital tools are increasingly used to organize everyday information and support routine decisions. Meal planning is one such task: users must decide what to prepare while considering the ingredients available, the time required, and the effort involved. In a home kitchen, ingredients may be spread across cupboards and refrigerators, and users may not immediately remember which dishes can be prepared from them. A small, searchable recipe database can make this decision easier by connecting available ingredients with stored recipe information.

Dishcovery is developed as an academic DBMS project to demonstrate how structured data and relational queries can support a practical everyday task. It is not intended to replace professional nutrition advice or commercial meal-planning platforms; its focus is on ingredient matching and database application concepts.

1.2 Identification of Problem

The central problem is the difficulty of identifying suitable recipes from a user's currently available ingredients while also understanding which ingredients are missing. A static list of recipes does not automatically compare a user's inventory with each recipe, and unstructured recipe notes make it difficult to retrieve and compare information consistently.

1.3 Identification of Tasks

Identify the application requirements and define the project scope.

Design a relational database for recipes, ingredients, categories, and their relationships.

Create and populate the MySQL tables with sample recipe data.

Develop a Flask application that connects the web interface to the database.

Build the ingredient selector, search results, sorting controls, and recipe details page.

Test database retrieval, matching logic, navigation, and the presentation of missing ingredients.

Prepare project documentation and a user manual.

1.4 Timeline

The work was completed in stages: requirement selection and project naming; database setup and sample data entry; Flask environment setup and database connectivity; development of the homepage and ingredient selection; implementation of recipe matching and details pages; interface refinement and sorting; testing, GitHub version control, and report preparation. Insert the actual dates from your work log if the supervisor requires a dated Gantt chart.

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks, and organization of the project. Chapter 2 reviews the background and compares common approaches to recipe discovery. Chapter 3 explains the selected features, constraints, design alternatives, architecture, and implementation methodology. Chapter 4 describes the implemented solution and validation tests. Chapter 5 presents the conclusion and possible future enhancements. References, appendices, and a user manual follow.

CHAPTER 2: LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The underlying need for recipe discovery is associated with everyday meal planning and household ingredient management. Traditional methods include handwritten recipes, cookbooks, personal memory, and informal lists. Digital recipe collections made it easier to search by dish name or category. Ingredient-based search further shifted the task toward finding recipes that use items already available. Dishcovery applies this general idea in a small academic database project, with emphasis on transparent matching and relational data handling rather than a large commercial recommendation engine.

2.2 Existing Solutions

Common approaches include printed cookbooks, general web searches, recipe websites, mobile recipe applications, and personal spreadsheets. Cookbooks offer curated instructions but require manual browsing. General search engines provide a broad range of recipes, although users may need to compare ingredient lists themselves. Recipe platforms can offer filters and recommendations, but their scope and interfaces may be more extensive than needed for a small classroom demonstration. A personal spreadsheet can store recipe information but usually requires manually maintaining relationships and calculating matches.

2.3 Comparative Feature Analysis

Approach	Ingredient matching	Structured storage	Main limitation
Cookbook / handwritten list	Manual	Low	Browsing and comparison take time
General web search	Depends on query	External pages	May require manual ingredient comparison

Spreadsheet	Possible with formulas	Moderate	Relationships and updates can become cumbersome
Dishcovery	Database-driven	Relational MySQL tables	Limited to its stored recipe dataset and current features

2.4 Review Summary

The comparison suggests that a focused system can be useful when the goal is to compare a known list of ingredients against a curated recipe database. The principal contribution of Dishcovery is not the invention of recipe search itself; it is the implementation of ingredient matching using a normalized relational structure and a simple web interface suitable for demonstrating DBMS concepts.

2.5 Problem Definition

Dishcovery must store recipe data consistently, allow a user to select available ingredients, retrieve recipes that overlap with the selection, calculate the number of matched and missing ingredients, and present the results in an understandable form. The project does not currently aim to generate recipes automatically, verify nutritional suitability, manage user accounts, or fetch live recipes from external services.

2.6 Goals / Objectives

Create a relational MySQL database for recipes, ingredients, categories, and mapping relationships.

Provide a simple browser-based interface for selecting and searching ingredients.

Use database queries to compare user selections with recipe ingredient requirements.

Display matched and missing ingredient counts, cooking time, difficulty, and readiness status.

Allow search results to be sorted by time, matched ingredients, or missing ingredients.

Provide a recipe details page with stored instructions.

Validate core application flows and document the design and operation.

CHAPTER 3: DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

The selected features prioritize usefulness, clarity, and database demonstration. Ingredient selection must be straightforward, so a searchable popup was chosen instead of displaying the full ingredient list permanently. Users can also add new ingredients directly through the interface. Newly added ingredients are stored in the MySQL database and become available for selection when adding recipes.

Results include matching counts and missing ingredients because these explain why a recipe is suggested. A separate details page keeps the search cards concise while providing the full recipe information when requested.

Feature	Reason for inclusion	Priority
Ingredient selector and search and addition	Simplifies ingredient selection and allows users to add ingredients that are not already stored in the database	High
Recipe matching	Core function of the application	High
Missing ingredient display	Makes results actionable	High
Recipe details page	Provides full stored instructions	High
Sorting options	Improves comparison of search results	Medium
Accounts and saved favourites	Useful extension but not necessary for the core demonstration	Future

3.2 Design Constraints

Time: the project is intended to be completed within a short academic project schedule.

Cost: the implementation uses a local development environment and free software tools.

Data scope: matching quality depends on the recipes and ingredients entered into the database.

Technical scope: the application uses a straightforward Flask–MySQL architecture rather than a distributed system.

Usability: the interface should be understandable to first-time users and should avoid unnecessary steps.

Security: database credentials must be kept outside public source control, and input should be handled safely.

3.3 Analysis of Features and Finalization Subject to Constraints

Features requiring external APIs, user authentication, personalized nutrition calculations, or live third-party recipe collection were excluded from the initial version because they would increase development time and introduce dependencies beyond the project's core learning goals. The final scope includes local database-backed search, result sorting, readiness labels, and recipe details.

3.4 Design Flow and Alternative Designs

Alternative A: a desktop application built with a graphical toolkit and connected to MySQL. This approach can be direct to implement but is less convenient to demonstrate through a browser and requires the interface to be run as a desktop window.

Alternative B: a Flask web application with HTML, CSS, and JavaScript connected to MySQL. This approach separates the user interface from server-side database logic and allows the project to be demonstrated in a browser. It was selected for Dishcovery.

3.5 Design Selection

The Flask web application was selected because it supports a simple browser-based demonstration, allows the interface to be refined with standard web technologies, and keeps database access on the server side. MySQL was selected for relational data storage and SQL query practice. The selected design is proportionate to the project's time, learning objectives, and data requirements.

3.6 Implementation Plan / Methodology

- Create the dishcovery database and define tables for Recipes, Ingredients, Categories, Recipe_Ingredients, and Recipe_Categories.
- Populate the tables with sample recipe, ingredient, and category records.
- Implement the ingredient management feature to allow users to add new ingredients through the interface and store them in the Ingredients table.
- Configure the Flask application and database connection using environment-based settings.
- Render the home page with ingredients retrieved from MySQL.
- Accept the user's ingredient selection and submit it to the Flask route.
- Query recipe relationships, calculate matches and missing ingredients, and return results.
- Render recipe cards, enable sorting, and link recipe names to the details route.
- Test core flows and refine the interface.

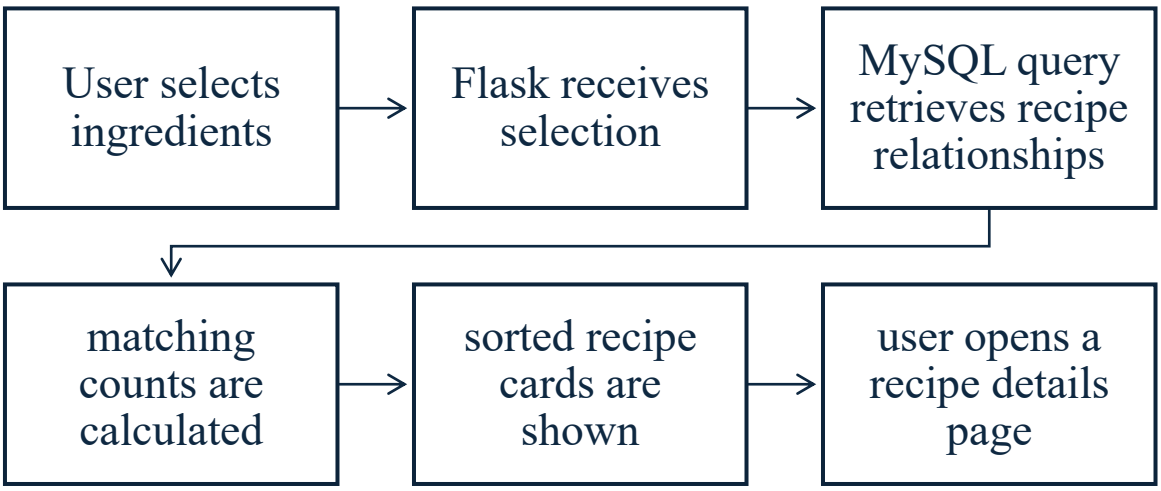


Figure 3.1: Implementation Diagram of Dishcovery

Entity–Relationship Diagram

Figure 3.3 presents the main entities in the Dishcovery database and the relationships between them. Recipes and Ingredients are connected through Recipe_Ingredients, while Recipes and Categories are connected through Recipe_Categories. These mapping tables allow one recipe to be associated with multiple ingredients or categories and allow an ingredient or category to be reused across multiple recipes.

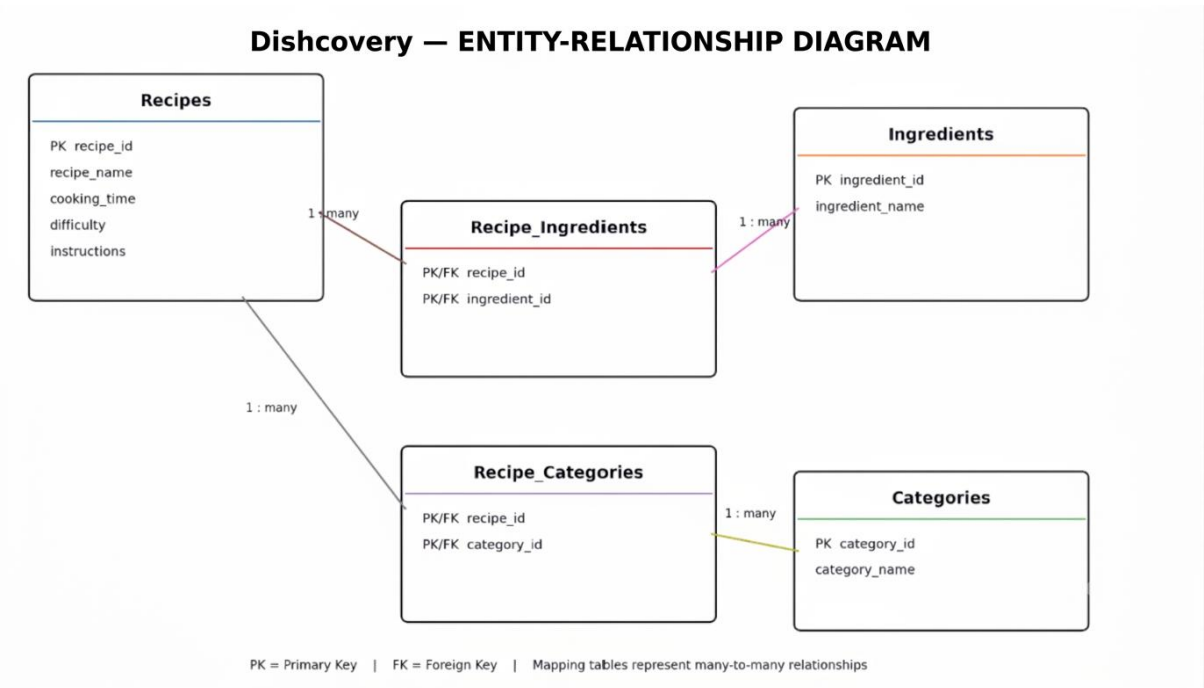


Figure 3.3: Entity–Relationship Diagram of Dishcovery

CHAPTER 4: RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

Dishcovery is implemented as a Flask application backed by a MySQL relational database. The browser presents the interface, JavaScript manages ingredient selection and result sorting, Flask handles routes and request processing, and MySQL stores the persistent recipe information. The separation of responsibilities keeps database operations on the server while allowing the interface to respond clearly to user actions.

Database Design

The database named dishcovery contains five main tables. Recipes stores recipe-level details such as name, cooking time, difficulty, and instructions. Ingredients stores reusable ingredient names. Categories stores category labels. Recipe_Ingredients links recipes with their required ingredients, and Recipe_Categories links recipes with categories. The mapping tables support many-to-many relationships: one recipe can use many ingredients, and one ingredient can appear in many recipes.

Table	Purpose	Relationship role
Recipes	Recipe name, time, difficulty, instructions	Parent entity for recipe mappings
Ingredients	Unique ingredient records	Parent entity for recipe-ingredient mappings
Categories	Recipe category names	Parent entity for recipe-category mappings
Recipe_Ingredients	Associates recipes and ingredients	Resolves many-to-many relationship
Recipe_Categories	Associates recipes and categories	Resolves many-to-many relationship

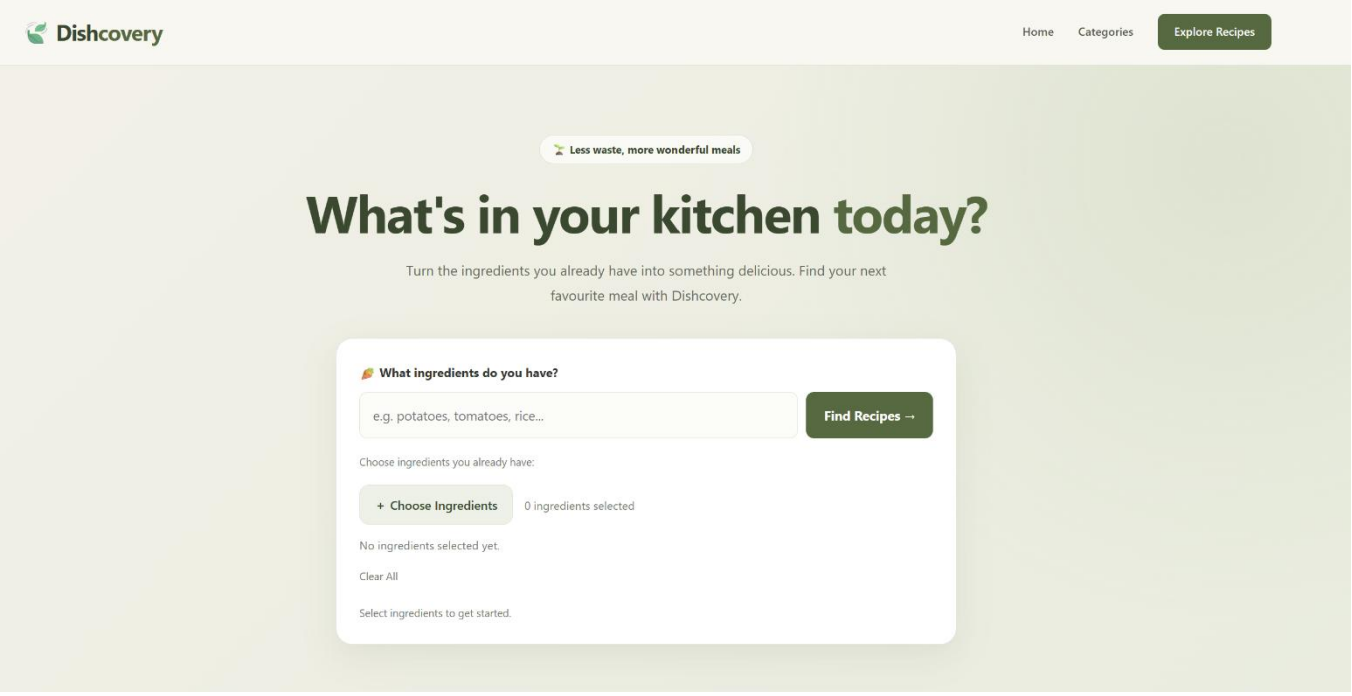
Application Workflow

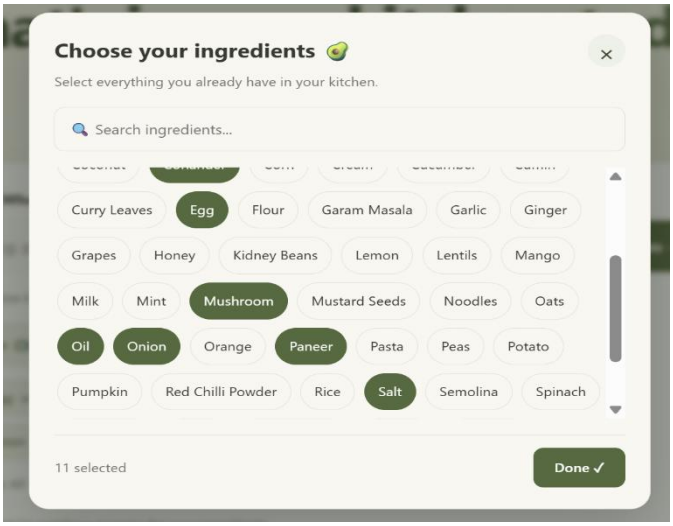
On the home page, ingredient options are loaded from the database. The user selects one or more ingredients in a popup and submits the selection. The application sends the selected values to the recipe-search route. SQL joins compare the selected ingredients with the ingredient mappings for each recipe. The result data includes matched ingredient count, total required ingredients, missing count, and missing ingredient names. JavaScript renders the recipe cards and allows sorting by cooking time, number of available ingredients, or number of missing ingredients. Selecting a recipe name opens the recipe details route. The ingredient

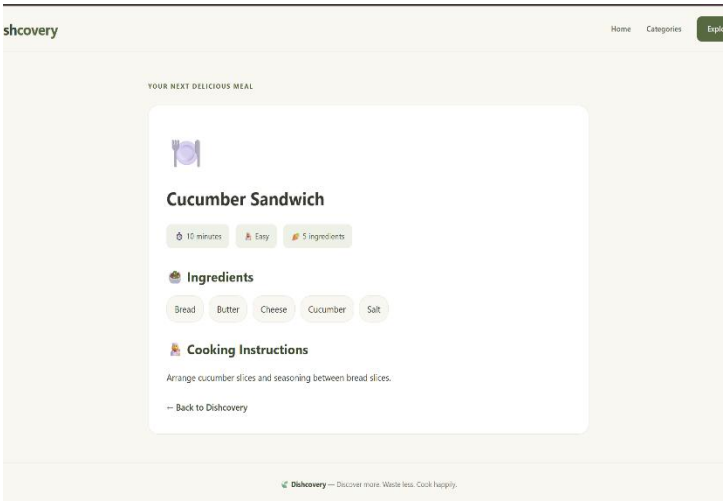
management feature was tested successfully. Newly entered ingredients were saved in the database and automatically selected in the ingredient-selection interface.

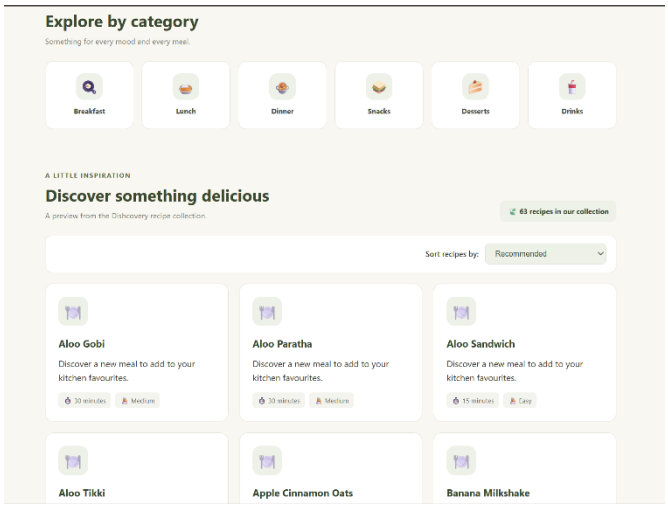
Validation and Test Cases

Test	Action	Expected result	Status
T1	Open home page	Ingredient selector and categories are visible	Passed in local testing
T2	Search ingredients	Matching recipes and ingredient counts are displayed	Passed in local testing
T3	Select a recipe name	Recipe details page opens	Passed in local testing
T4	Change sorting option	Results reorder according to selected option	Passed in local testing
T5	Remove or clear ingredients	Selected ingredient display updates	Passed in local testing
T6	Search with no selected ingredients	User receives a prompt to select ingredients	Verify before final submission









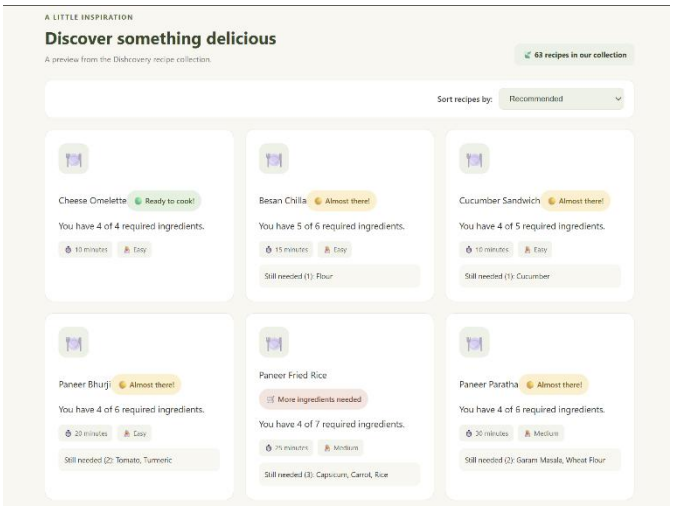


Table	Column	Type	Default Value	Nullable	Character Set	Collation	Privileges	Extra	Comments
categories	category_id	int		NO			select,insert,update,references	auto_increment	
categories	category_name	varchar(50)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
ingredients	ingredient_id	int		NO			select,insert,update,references	auto_increment	
ingredients	ingredient_name	varchar(100)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipe_categories	recipe_id	int		NO			select,insert,update,references		
recipe_categories	category_id	int		NO			select,insert,update,references		
recipe_ingredients	recipe_id	int		NO			select,insert,update,references		
recipe_ingredients	ingredient_id	int		NO			select,insert,update,references		
recipes	recipe_id	int		NO			select,insert,update,references	auto_increment	
recipes	recipe_name	varchar(100)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipes	cooking_time	int		NO			select,insert,update,references		
recipes	difficulty	enum('Easy','Mediu...		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipes	instructions	text		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipes_backup	recipe_id	int	0	NO			select,insert,update,references		
recipes_backup	recipe_name	varchar(100)		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipes_backup	cooking_time	int		NO			select,insert,update,references		
recipes_backup	difficulty	enum('Easy','Mediu...		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		
recipes_backup	instructions	text		NO	utf8mb4	utf8mb4_0900_...	select,insert,update,references		

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

Dishcovery demonstrates how a relational database can support a practical recipe discovery task. It stores recipes, ingredients, and categories in related tables and uses SQL-based retrieval to compare user-selected ingredients with recipe requirements. The web interface displays the number of ingredients available, the ingredients still needed, cooking time, difficulty, and a readiness label. Users can sort results and open a dedicated page to view recipe information and instructions.

The project meets its principal learning objectives by combining database design, relational relationships, SQL joins, server-side request handling, and a browser-based interface. It also demonstrates the value of separating reusable ingredient records from recipe records and representing many-to-many relationships through mapping tables. The current results are dependent on the completeness and accuracy of the sample data. The application does not currently provide automatic recipe generation, nutrition verification, or personalized recommendations.

During validation, the main user flows—including ingredient selection, recipe search, result sorting, and opening recipe details—were confirmed to work in the local development environment. Final submission should include current screenshots and the results of a last test run.

5.2 Future Work

Add user accounts and allow users to save favourite recipes.

Introduce filters for dietary preferences, cuisine, cooking time, and difficulty.

Add recipe ratings and feedback to support better discovery.

Improve ingredient matching to handle synonyms, spelling variants, and quantities.

Provide an option to prioritize recipes that use the greatest proportion of available ingredients.

Add nutrition information only from a reliable and appropriately attributed data source.

Expand and validate the recipe dataset, including clear instructions and consistent ingredient naming.

Deploy the application securely so it can be accessed beyond the local development environment.

REFERENCES & APPENDIX:

List only the sources actually consulted during development and report preparation. The following official documentation is relevant to the technologies used:

1. Python Software Foundation. Python Documentation. https://docs.python.org/3/

2. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

3. Oracle. MySQL Reference Manual. https://dev.mysql.com/doc/

4. MDN Web Docs. HTML: HyperText Markup Language. https://developer.mozilla.org/en-US/docs/Web/HTML

5. MDN Web Docs. CSS: Cascading Style Sheets. https://developer.mozilla.org/en-US/docs/Web/CSS

6. MDN Web Docs. JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

USER MANUAL

1. Purpose

This manual explains how to run Dishcovery in a local development environment and use its main features.

2. Prerequisites

Python installed on the computer.

MySQL Server installed and running.

The Dishcovery project folder and required Python packages.

Correct local database settings stored in the app's environment configuration.

3. Starting the Application

Start MySQL Server and confirm that the dishcovery database and its tables are available.

Open the Dishcovery project folder in a terminal or code editor.

Activate the project's virtual environment, if one was created.

Install project dependencies from the project's requirements file if needed.

Check the local environment configuration for the database host, database name, username, and password. Do not publish these values.

Run the Flask application using the command configured for the project (for example, run app.py from the app directory if that is how the project is set up).

Open the local address printed by Flask in a web browser.

4. Finding Recipes

On the home page, click “Choose Ingredients”.

Use the search field in the popup to find an ingredient.

Click each ingredient you currently have; selected ingredients are visually highlighted.

Click “Done” to close the popup. Review the selected ingredient chips on the home page.

Click “Find Recipes” to search the database.

Review the recipe cards, which show ingredient availability, missing ingredients, cooking time, difficulty, and readiness status.

Use the sorting dropdown to order results by recommendation, quickest cooking time, most ingredients available, or fewest missing ingredients.

Click a recipe name to open its details page and read the stored ingredients and cooking instructions.

5. Managing Selection and Troubleshooting

To remove one ingredient, use the × control on its selected chip. To remove all selected ingredients, click “Clear All”. If the application cannot search, confirm that Flask and MySQL are running, the database connection settings are correct, and the required tables contain data. Refresh the page and try again. If a recipe is not shown as expected, check that the recipe and its ingredient relationships are present in the database.

GitHub repository: https://github.com/ashdotcom29/Dishcovery.git